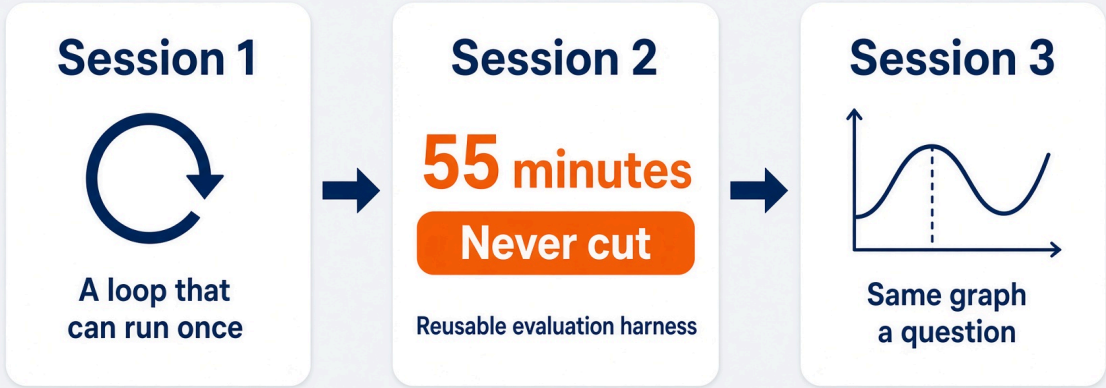
Harness Engineering, the validation layer

Session 2. The center of gravity. 55 minutes.

Saturday 29 August 2026. 11:10 Central.

Rick Hightower. Spillwave. Packt workshop.

55 minutes. This is the one that does not get cut.

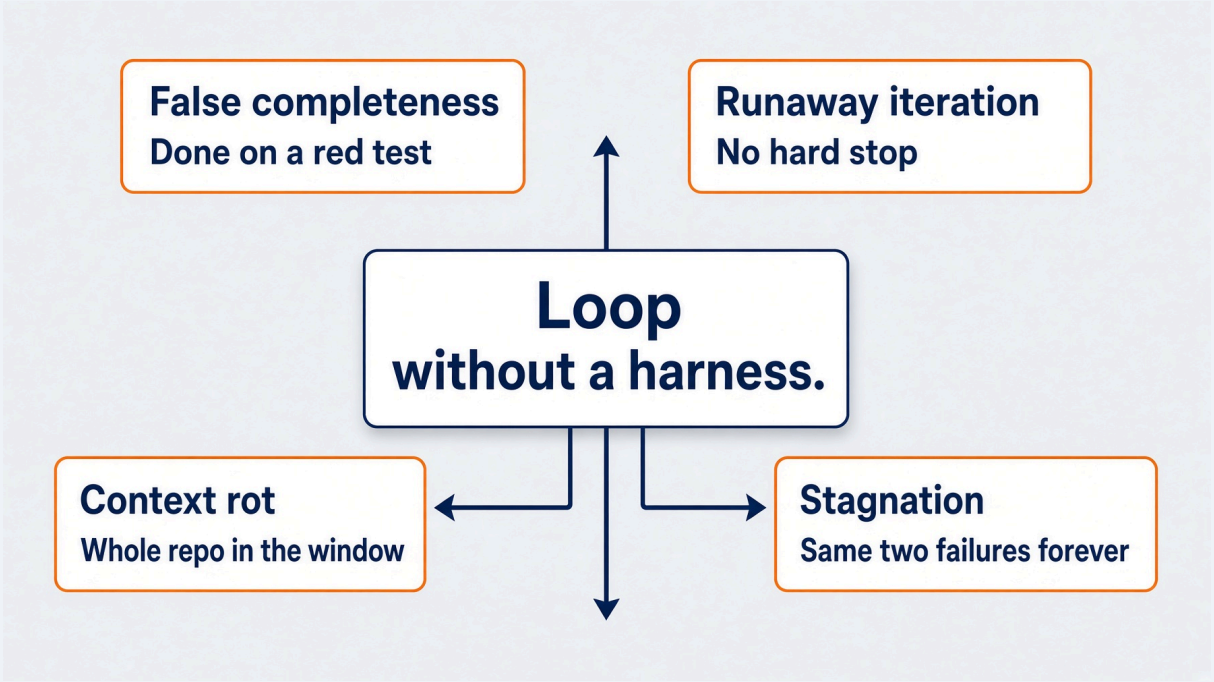


BLOCK	MINUTES	WHAT THEY KEEP
Why loops fail without a harness	0 to 15	Maker and Checker as types
Spec-driven development	15 to 25	Intent as a testable contract
Lab. Fill `harness.py`	25 to 50	Three functions that hold the loop
Reading output. When to stop	50 to 55	Three exits, a receipt, a gate

The loop you just built will lie to you.

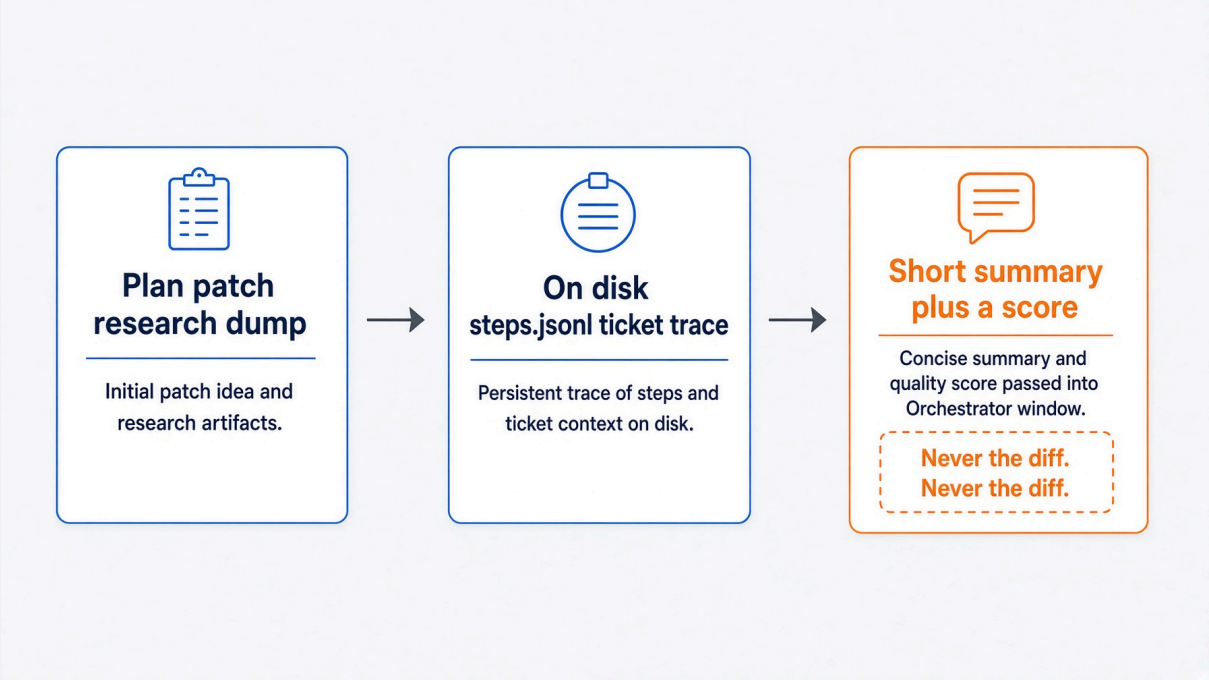
- It can edit forever.
- It can declare victory on a red test.
- It can stuff the whole repo into the next call.
- A harness is how you stop that. Not a better prompt.

Four ways a loop lies. Each is a missing harness piece.



FAILURE	MISSING PIECE	WHAT THIS HOUR ADDS
False completeness	Independent verify	Ten-row rubric, red gate
Runaway iteration	External transition	`gates.decide()`
Context rot	Memory on disk	Planner subagent, summaries
Stagnation	Progress detection	Same signature twice

Context is not memory. The window is a scratch pad.



The orchestrator sees summaries. Never the whole plan. Never the patch.

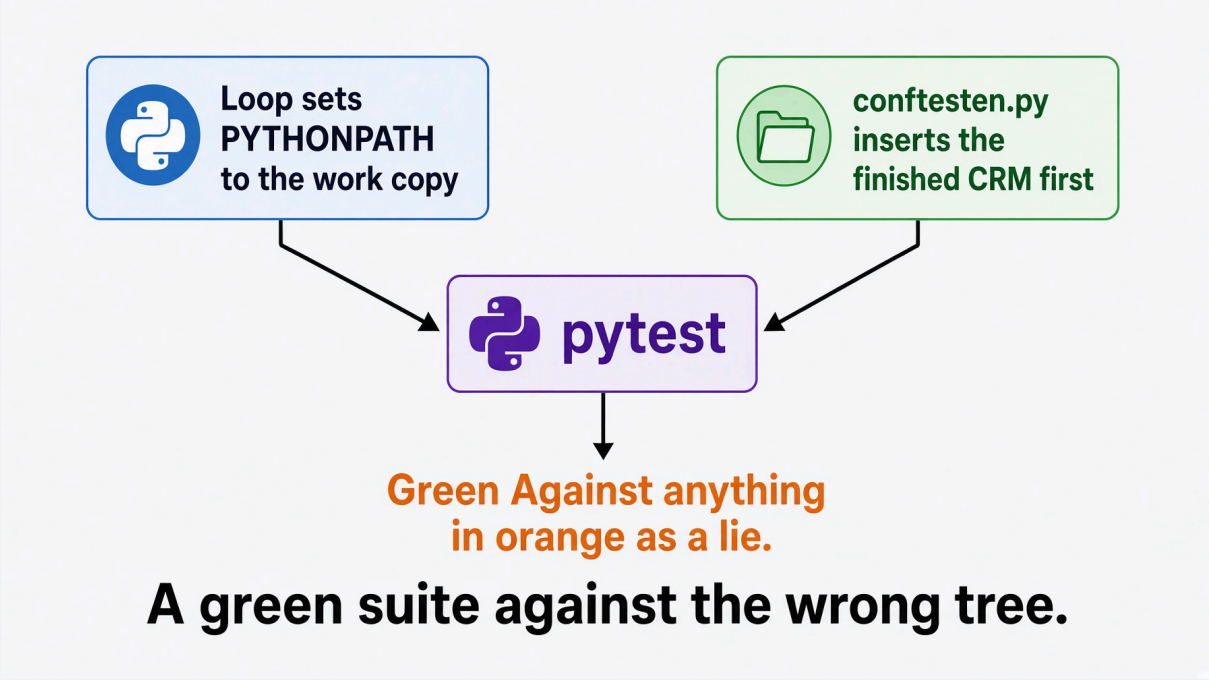
That is why the planner runs as its own subagent. It writes `steps.jsonl` and returns a count.

Liu et al., Lost in the Middle. TACL 2024. arXiv:2307.03172

A true story from this repo.

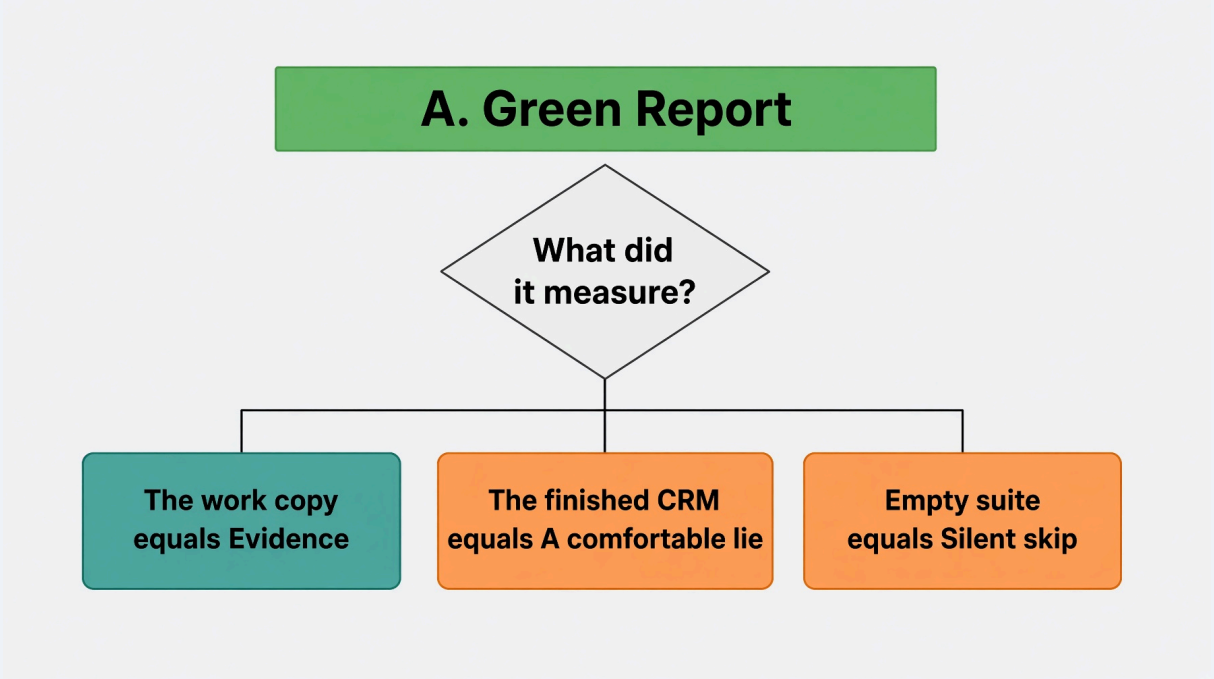
Seven tests. Green on every run. They had never tested the thing they named.

```
# tests/conftest.py
CRM_ROOT = Path(__file__).resolve().parents[2] / "crm"
sys.path.insert(0, str(CRM_ROOT))   # wins over the PYTHONPATH the loop sets
```



The loop pointed the tests at the work copy. The conftest pointed them at the finished answer.

A check that measures the wrong thing is worse than no check.



The fail-then-pass demo had never once worked. Nothing reported an error.

Absent evidence is never clean. That rule shows up in the rubric, the receipt, and the final judge.

Maker and Checker

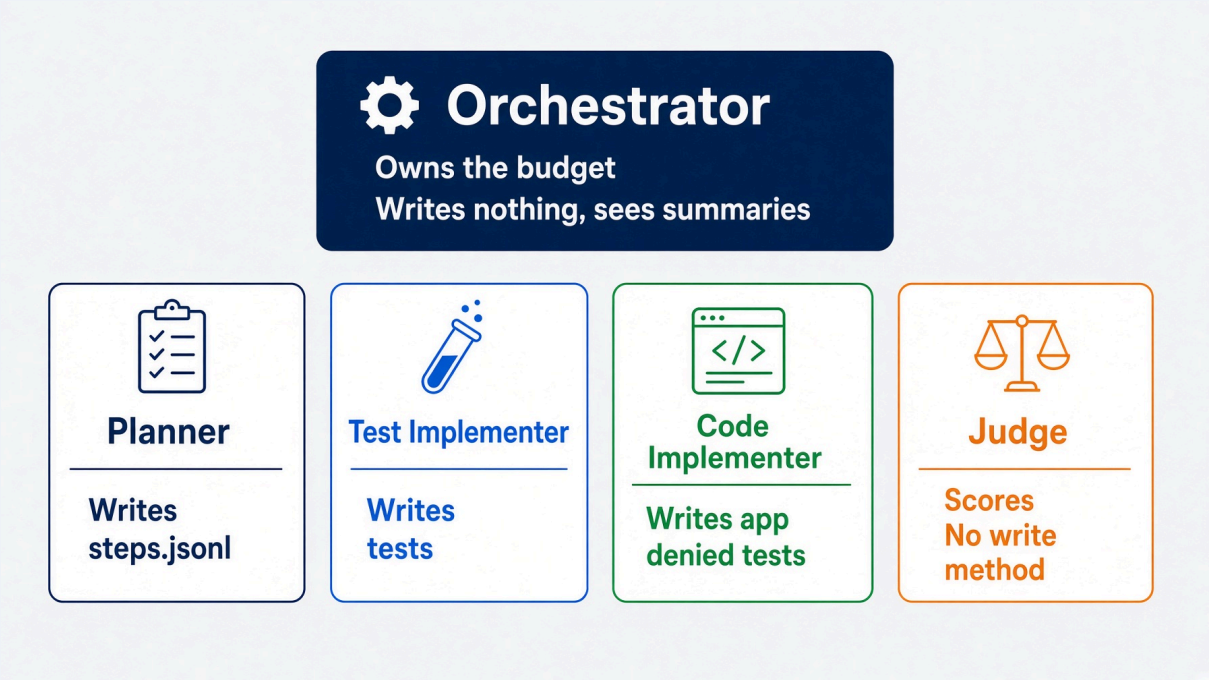
Two doers. Disjoint scope. A judge with no hands.

Two doers. Disjoint scope.

- The test implementer writes `tests/**`. Nothing else.
- The code implementer writes `app/**`. It is **denied** `tests/**`.
- The judge writes nothing at all.

The code implementer cannot weaken a test to reach green. Not because it was told not to. Because it holds no write path to one.

Five roles. Write scope is the point.



`loops/roles.py` · `build()`

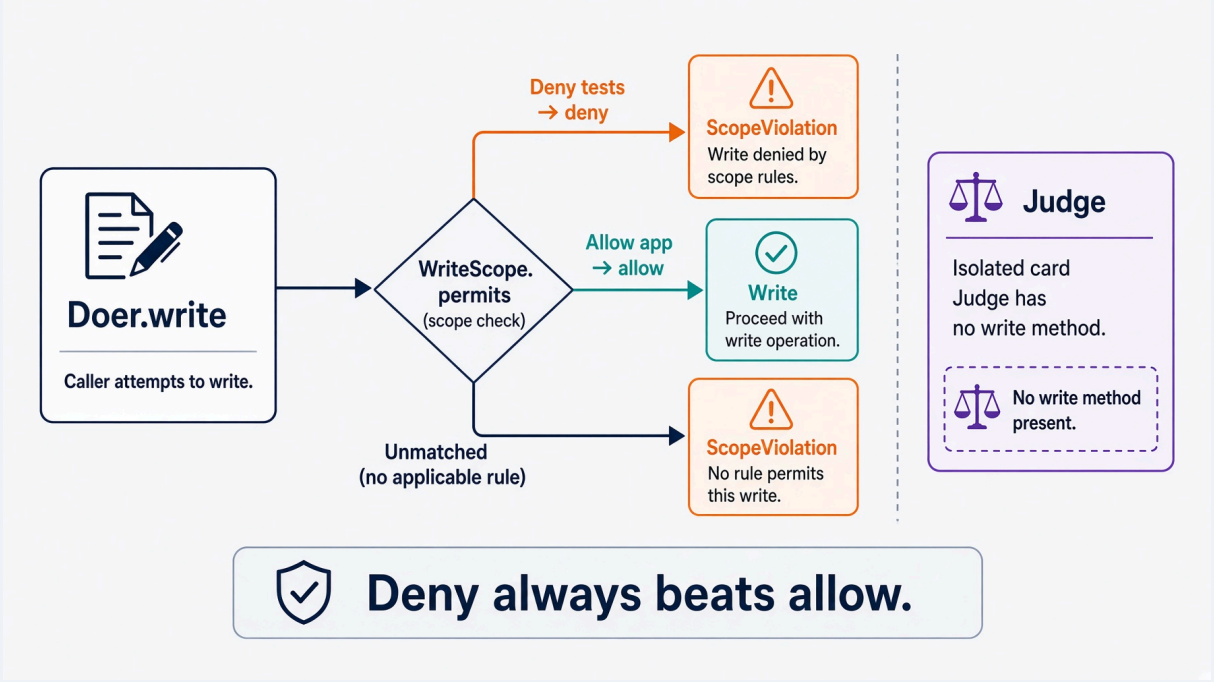
Scope is a type, not a sentence.

```python
@dataclass
class Judge(Role):
    """Scores work. Holds no write path.

    There is deliberately no `write` method on this class.
    Adding one is not a convenience. It is the end of the split.
    """

    def read(self, relative: str) -> str:
        return (self.repo / relative).read_text(encoding="utf-8")
```

A rule in a prompt is a suggestion an agent can reason its way around. A missing method is not.

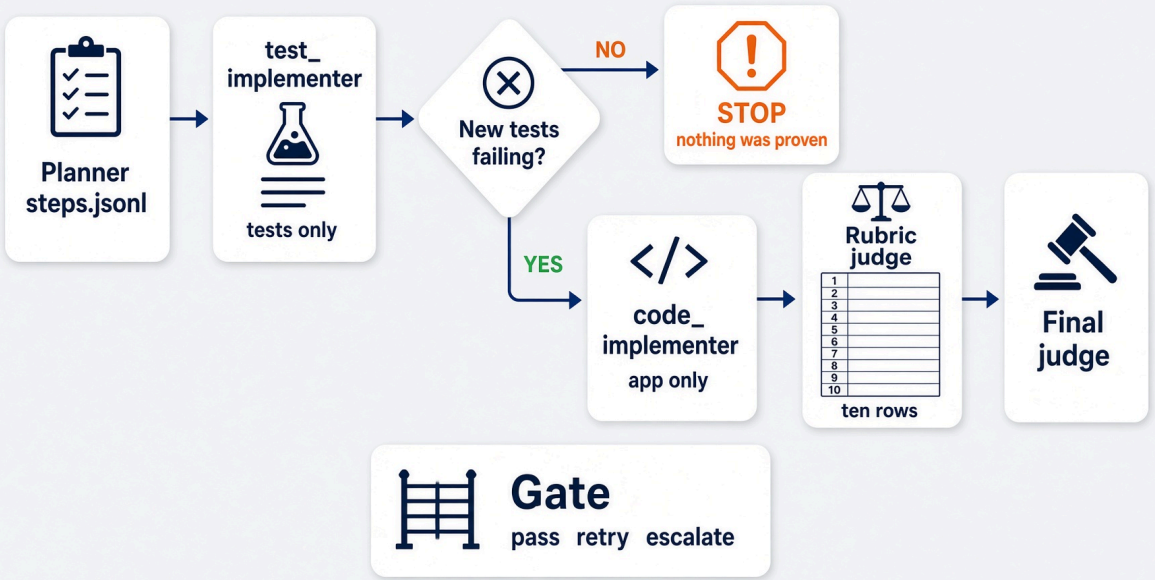
`loops/roles.py` · `test_a_judge_has_no_write_method`

Deny always beats allow.




```
code_implementer:
  write_allow: ["app/**", "src/**"]
  write_deny:  ["tests/**"]
```

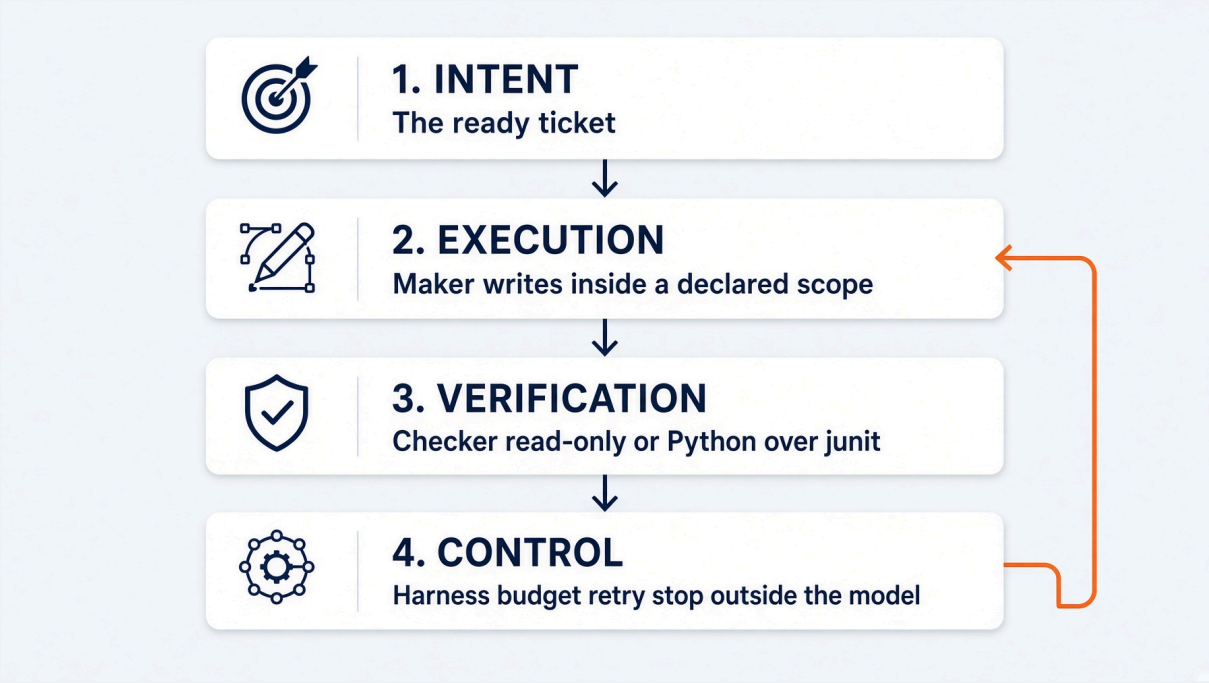
`loops/roles.py` · `WriteScope.permits` · `.loop.yml` in the target



The orchestrator owns the budget and sees summaries, never the diff.

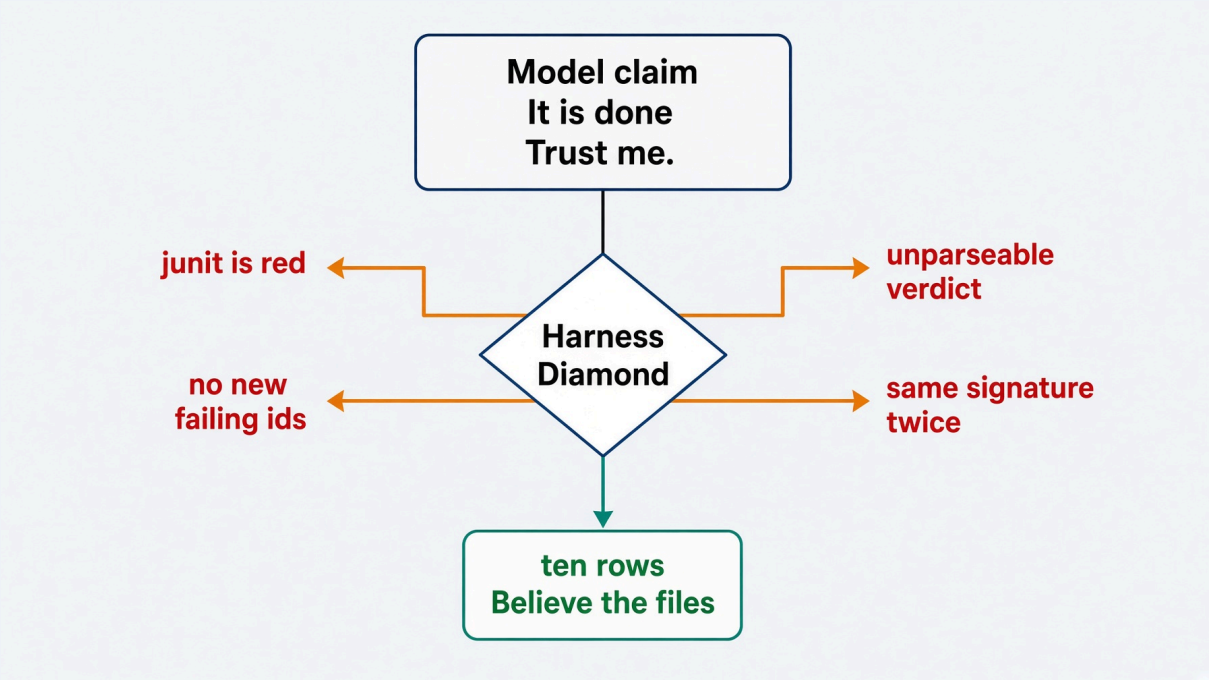
Python holds the loop. The model never counts its own retries.

Four planes. One graph.



PLANE	WHO	JOB
Intent	The ready ticket	A contract a test can fail
Execution	Maker	Writes inside a declared scope
Verification	Checker	Read-only, or Python over junit
Control	Harness	Budget, retry, stop. Outside the model

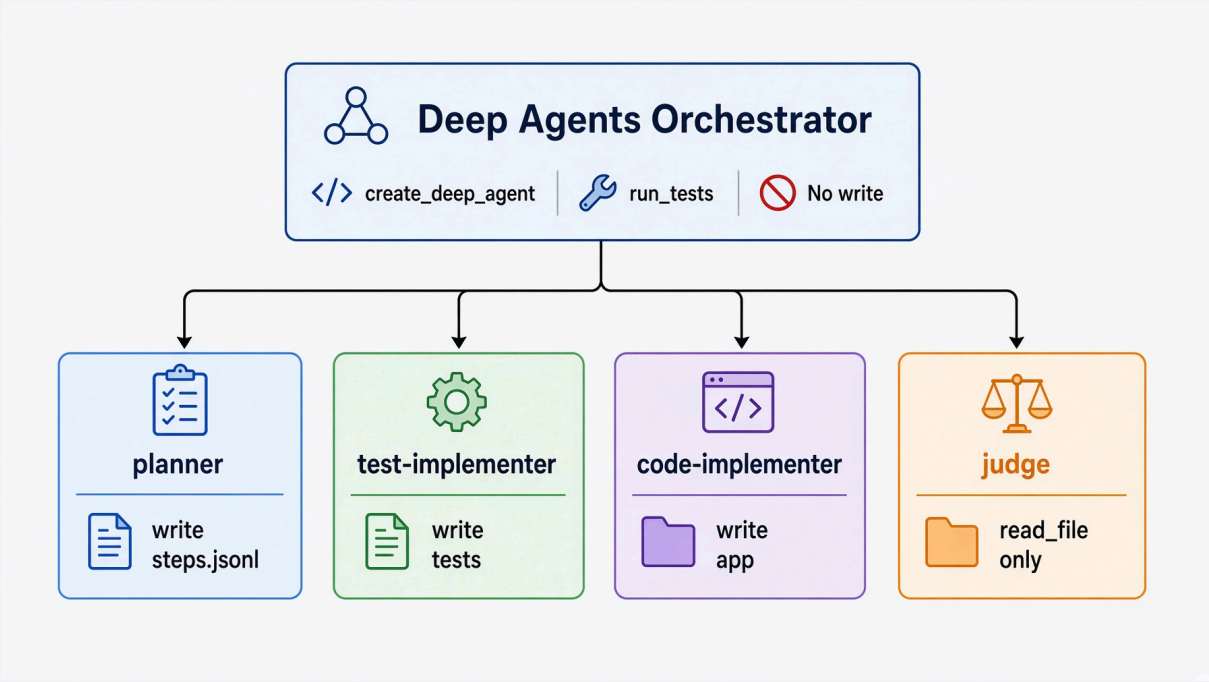
Hallucination containment is a missing method, not a better prompt.



The model proposes. The harness decides. Claims are not evidence. Files are.

This graph in Deep Agents.

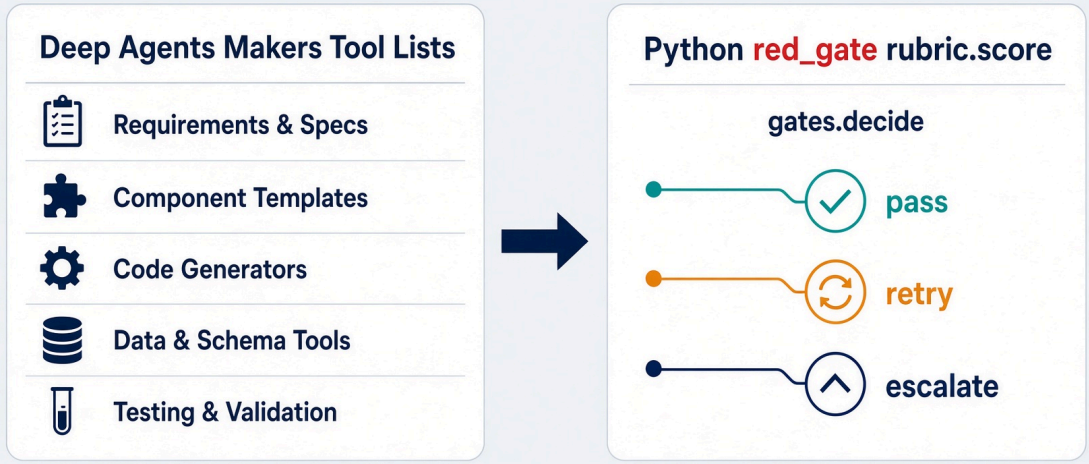
`create_deep_agent` is a harness, not a chat wrapper.



- Each subagent gets its own `tools` list. That list **replaces** the parent.
- The judge's list is `read_file`. No `write_file`.
- Path scope lives inside the write tool. `WriteScope.check` still runs.

`solutions/sol2_implementer_deep_agents/roles.py`

Python owns the gate. `create_deep_agent` does not count retries.



Python owns the gate.

```
return create_deep_agent(
    model=model,
    tools=[run_tests_tool(repo)],
    subagents=subagents_for(contract, loop),
)
```

Saturday lab stays Claude Code. Fill `harness.py`.

The Deep Agents port is the takehome. `solutions/sol2_implementer_deep_agents/`. Issue 118.

Spec-driven development

Intent becomes a contract the machine can check.

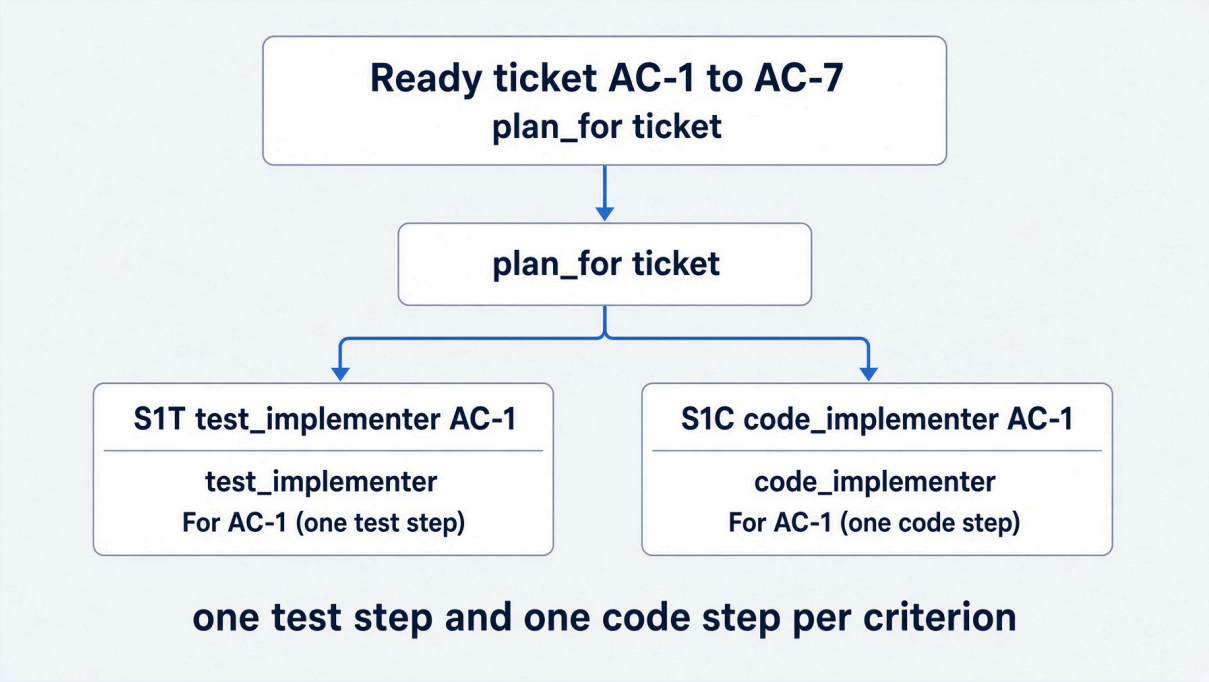
If an acceptance criterion cannot fail a test, it is a wish.

```
- (AC-4) `GET /api/tasks?due_before=<date>`
  returns only tasks due before that date,
  and never tasks with no due date.
```

Seven acceptance criteria. Each one names a condition a test can be red about.

"Should be intuitive" names nothing.

Graph engineering. Intent becomes a graph of steps.



`loops/implementer.py` · `plan_for()`. Derived, not generated. Swapping this for a planner subagent is the stretch. The schema it must satisfy is already enforced.

The plan is a file, and it is checkable.

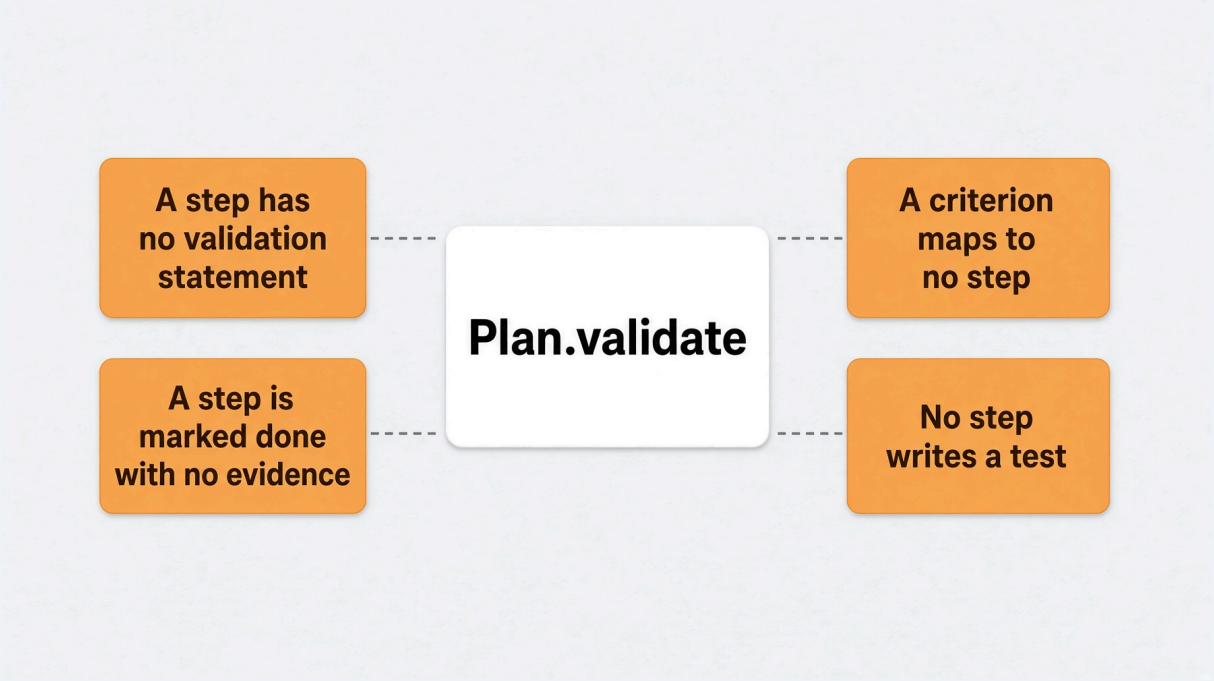
```
{"id":"S1T","ticket":"T001","role":"test_implementer",
 "action":"Write a test that fails until due_date accepts null",
 "validation":"a test covering AC-1 exists and fails before any code",
 "criterion":"AC-1","status":"todo","evidence":null}
```

`steps.jsonl` . JSON Lines. Every step carries a **validation statement**.

The file is disposable. It belongs to one run against one ticket.

`loops/steps.py`

The plan is rejected when it cannot be checked.



`PlanRejected`. The orchestrator refuses to run it.

Marking a step done without naming the test that proves it is grading the loop on a claim.

`loops/steps.py` · `validate()` · `mark()`

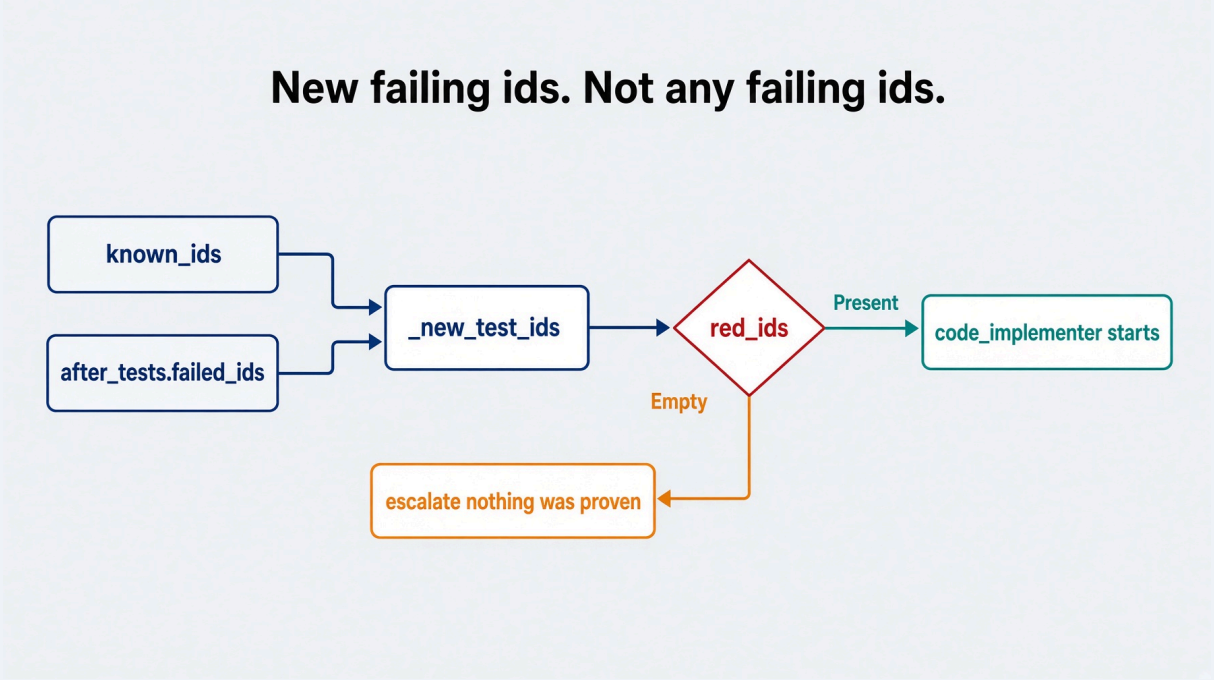
The red gate. Tests first, and prove it.

1. The test implementer writes the tests.
2. The orchestrator reads `junit.xml`.
3. **If the new tests are not failing, stop.**

A test that passes before any code exists proves nothing. It is the most comfortable kind of nothing, because it is green.

New failing ids. Not any failing ids.

```python
def _new_test_ids(before: set[str], after_failed: set[str]) -> set[str]:
    """Test ids that are failing now and did not exist before. The red proof."""
    return {test_id for test_id in after_failed if test_id not in before}
```



`loops/implementer.py` · `_new_test_ids` · `require_red` in `.loop.yml`

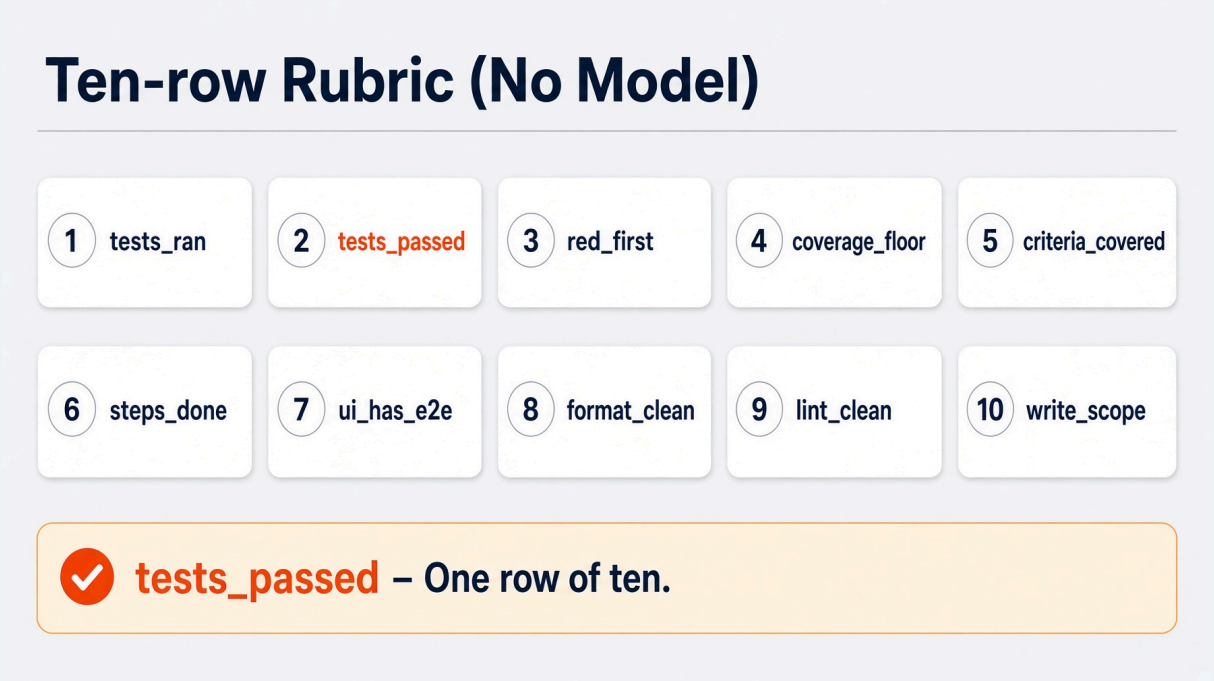
The rubric. Ten rows. No model call.

```
PASS  tests_ran          25 tests
PASS  tests_passed       all green
PASS  red_first          6 tests were red first
PASS  coverage_floor     80.42% against a floor of 78.0%
PASS  criteria_covered   all covered
PASS  steps_done         14 steps: done 14
PASS  ui_has_e2e         2 interface files changed, end-to-end green
PASS  format_clean       clean
PASS  lint_clean         clean
PASS  write_scope        every write was inside its role's scope
```

Every row is computed from `junit.xml`, `coverage.xml`, exit codes, `steps.jsonl`, and the diff.

`loops/rubric.py` · `score()`

"The tests passed" is one row of ten.



A judge that checks only `tests_passed` can be satisfied by an agent that writes one trivial test and deletes the rest.

Absent evidence is never a pass. Every argument left `None` becomes a failing row.

Two judges, and only one of them guesses.

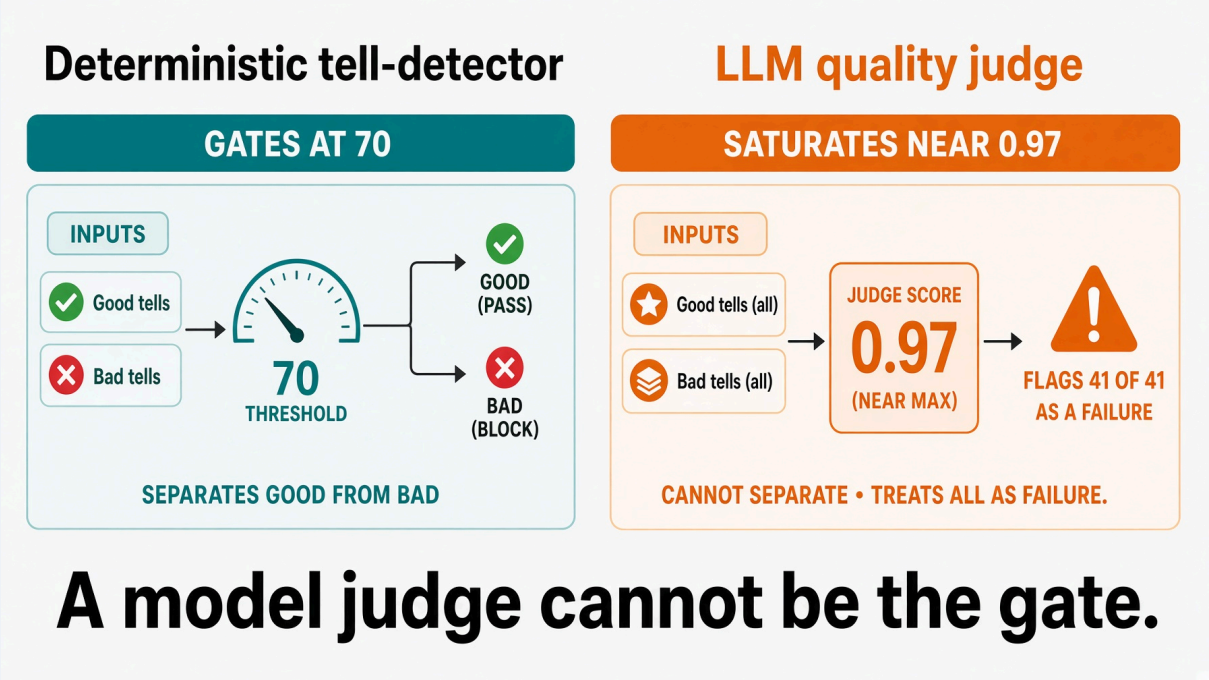
QUESTION	WHO ANSWERS
Did the tests run and pass?	Arithmetic
Does coverage meet the floor?	Arithmetic
Did anyone write outside scope?	Arithmetic
Is the ticket actually done?	A model

Use a model only where you must.

The implementer's judge is deterministic. The enhancer's judge reads ticket prose and needs a model.

Why a model judge cannot be the gate.

Measured across 41 published articles in a production pipeline:

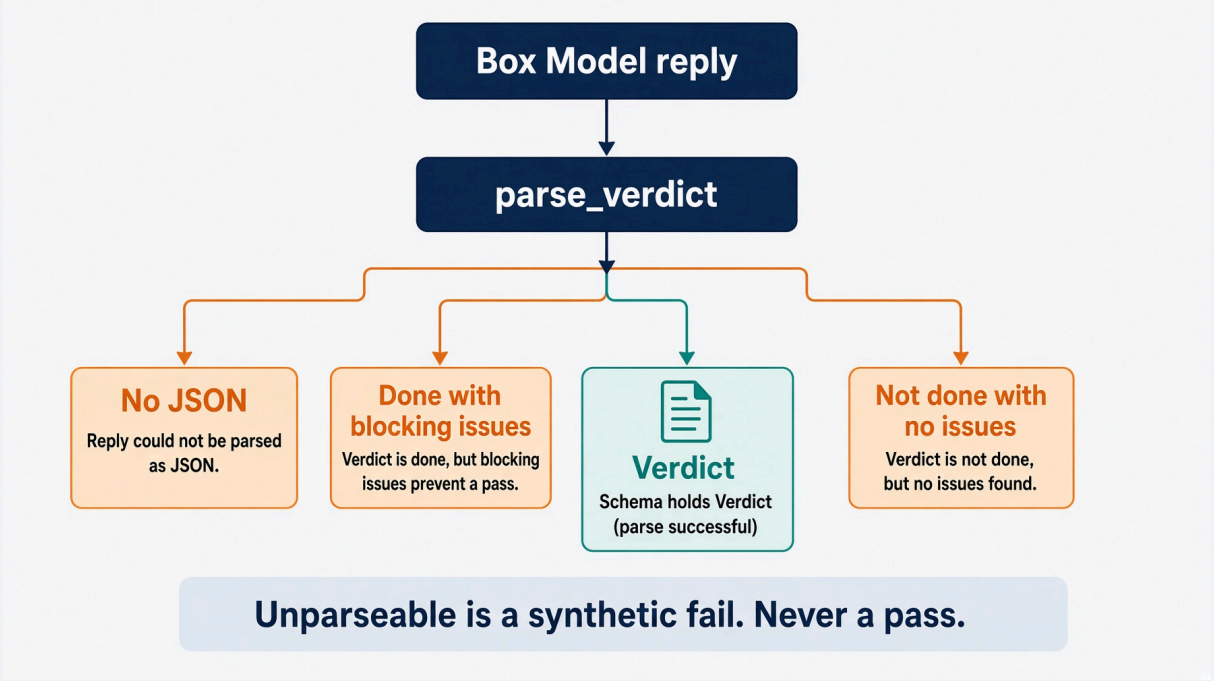


JUDGE	BEHAVIOR
Deterministic tell-detector	Gates at 70. Separates good from bad.
Large Language Model (LLM) quality judge	Saturates near 0.97. Flags 41 of 41, regardless.

A judge that approves everything is not a judge. It is a rubber stamp with a temperature setting.

So make the model's verdict a schema.

```
if verdict.done and verdict.blocking_issues:
    return synthetic_fail("says done while listing blocking issues")
```



- A pass carrying a critical issue is not a decision. Reject it.
- Output that will not parse is a FAIL, never a pass.
- Absent evidence is never clean.

`loops/final_judge.py` · `parse_verdict` · `synthetic_fail`

The gate that makes it real.

```
BLOCKED by pre-tool hook: git push
Last run: FAILED (1 tests).
  first failure: tests.test_due_date::test_model_has_optional_due_date
Run `task test` first.
```

A Claude Code `PreToolUse` hook refuses `git push` without a green receipt.

No push and no pull request until the suite runs green locally.

Your agent will hit this today.

Lab 2. The Ticket Implementer

25 minutes. A ready ticket in. A green rubric out.

Lab. 25 minutes. Fill `harness.py`.

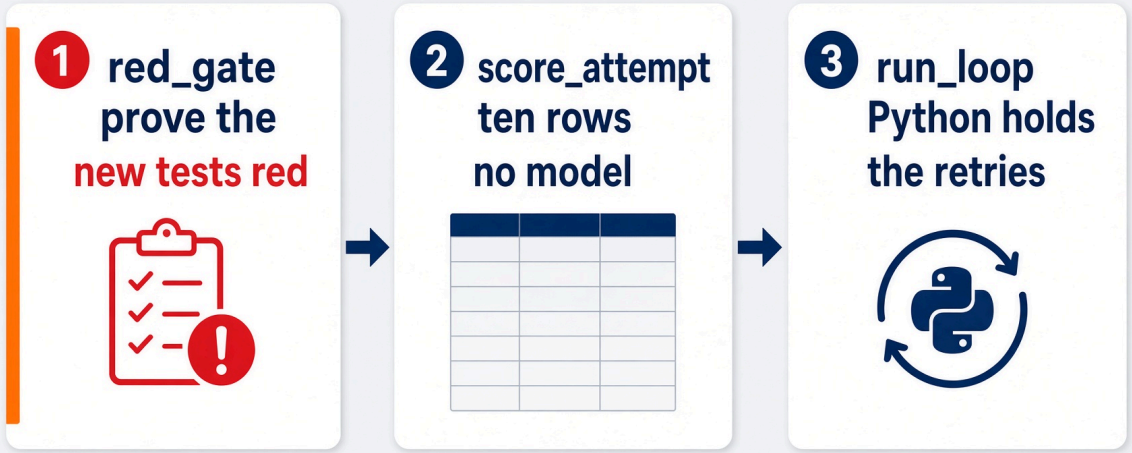
```
cd labs/lab2_implementer
claude -p "$(cat prompts/claude-code.md)"
```

Fill three functions. Nothing else.

```
task loop:implementer -- --ticket T001 --doer reference   # ten rows
task loop:implementer -- --ticket T001 --doer none        # red gate refuses
```

Falling behind is fine: watch Rick finish `harness.py` and keep going.

Three functions. The order is the lesson.




```
tests first  ->  prove them red  ->  code until green  ->  judge  ->  gate
```

`red_gate` . `score_attempt` . `run_loop` . The stub already imports `gates` and `rubric` .

`labs/lab2_implementer/harness.py`

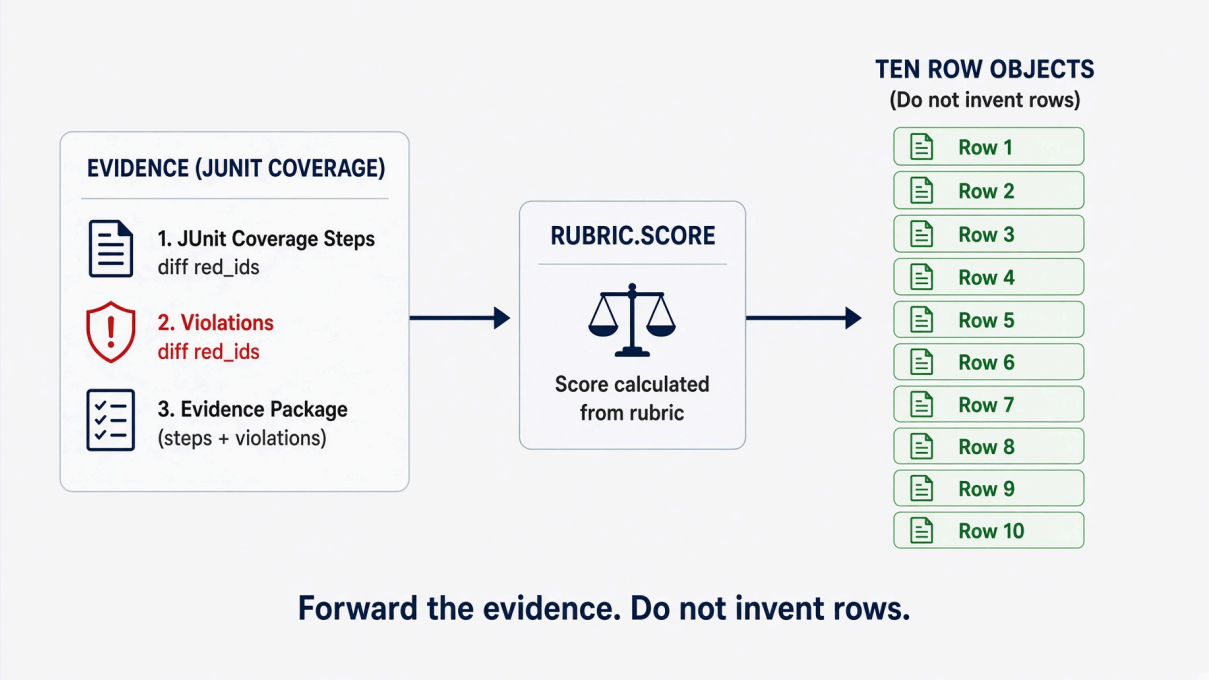
`red_gate(before, after)`. New ids that are failing now.

```
def red_gate(before: RunResult, after: RunResult) -> set[str]:
    """Return the test ids that are failing now and did not exist before.

    A test that passes before any code is written proves nothing,
    so an empty result must stop the loop rather than let it continue.
    """
    seen = before.junit.passed_ids | before.junit.failed_ids
    return implementer._new_test_ids(seen, after.junit.failed_ids)
```

An empty set is not a small problem. It is the red gate refusing.

`score_attempt`. Forward the evidence. Do not invent rows.

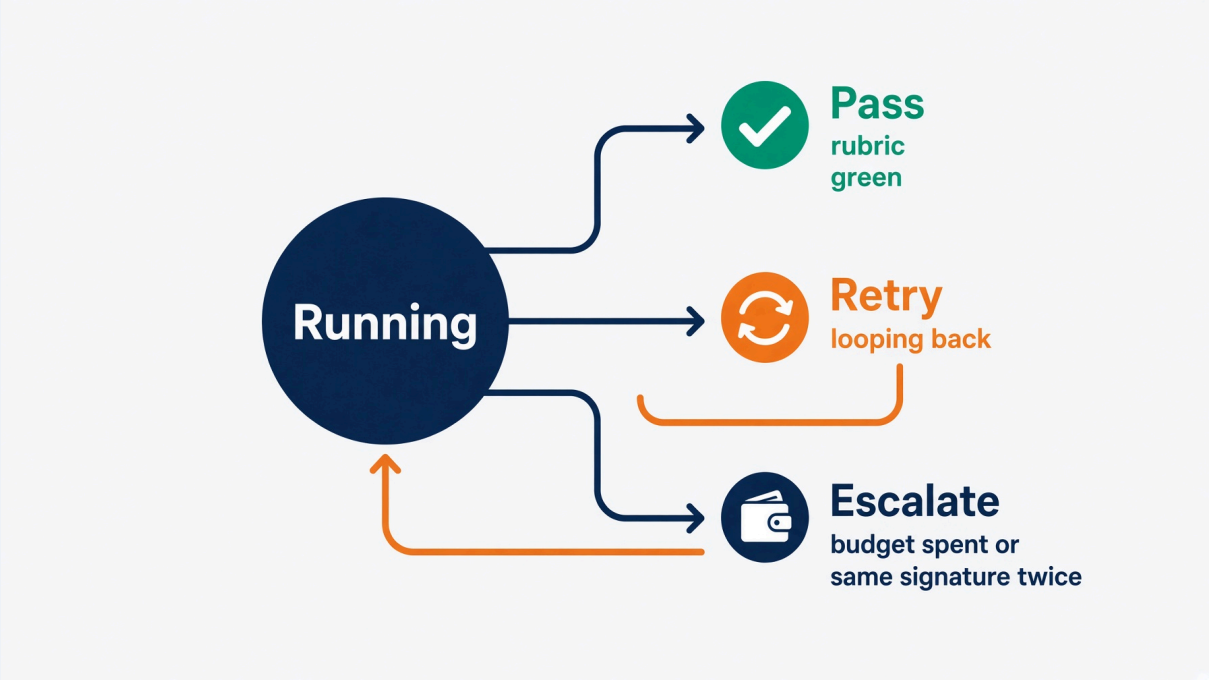
```
def score_attempt(contract: Contract, **evidence) -> rubric.Score:
    """Score one attempt against the ten rubric rows.

    Every argument left out becomes a failing row.
    Absent evidence is never a pass.
    """
    return rubric.score(contract=contract, **evidence)
```



Do not compute the rows yourself. `loops/rubric.py` already does.

`run_loop` . Python holds the retries.

```python
def run_loop(contract: Contract, budget: int = 3, ticket_id: str = "T001") -> dict:
    """Run the harness until it passes, stalls, or runs out of budget.

    Hold the loop in Python. The model does not get to count its own retries.
    """
    return implementer.run(
        repo=contract.repo,
        ticket_id=ticket_id,
        budget=budget,
        doer="reference",
    )
```



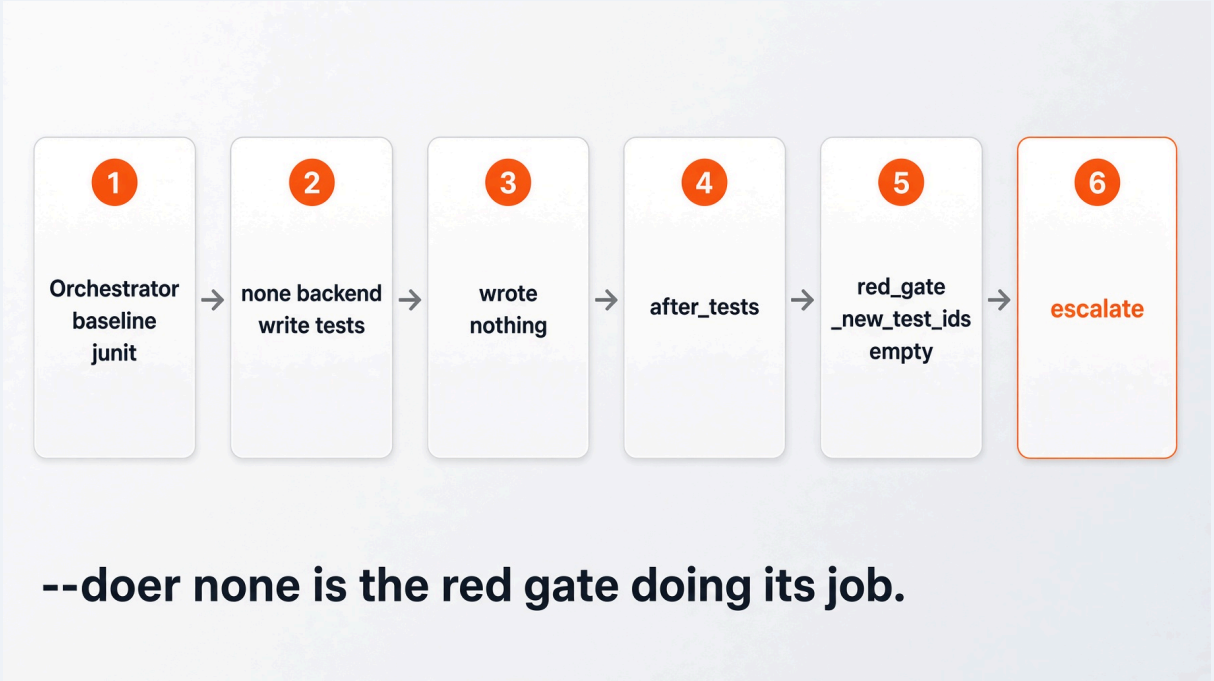
`loops/gates.py` · `decide()`

Two commands. Honesty is the second one.

```
task loop:implementer -- --ticket T001 --doer reference
# copies known-good into tests/** then app/**
# expect ten PASS rows and gate: pass

task loop:implementer -- --ticket T001 --doer none
# writes nothing on purpose
# expect gate: escalate
# reason: red gate: no new test was observed failing
```

The `none` backend is how you prove the loop reports failure honestly, with no model key.

`--doer none` is the red gate doing its job.



`test_a_doer_that_writes_nothing_is_stopped_by_the_red_gate`.

If this run were green, the harness would be lying.

Falling behind is fine. Watch Rick finish and continue.

```
cp harness.py harness.py.my-attempt
```

There is no drop-in `harness.py`. Watch Rick finish and type what he typed.

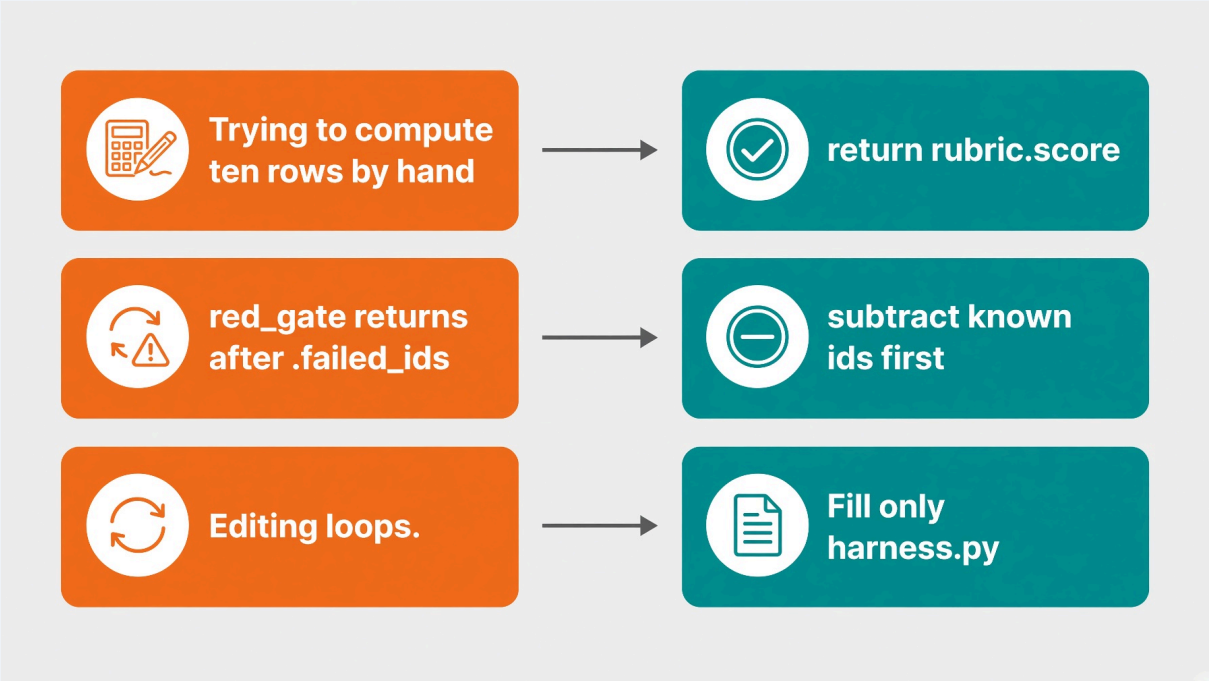
You continue the next module with a working artifact.

Put the empty stub back later if you want to retry:

```
git checkout -- harness.py
```

See `labs/lab2_implementer/FALL-BEHIND.md`.

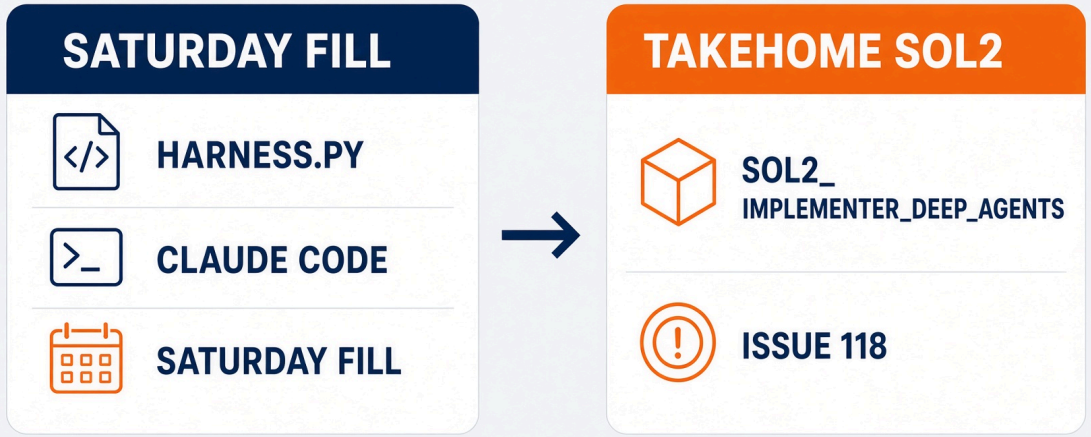
The common stall is `score_attempt`.



If you stall, read `loops/implementer.py`, `loops/rubric.py`, and `loops/gates.py`. They are the answer, not a hint.

Do not edit the target repo's tests to make something pass.

After class. The same eight steps on Deep Agents.




```
cd solutions/sol2_implementer_deep_agents
python3 -m pytest tests -q
python3 harness.py --repo ../../work/northwind-field-crm --ticket T001 --doer deep
```

`--doer deep` needs `deepagents` installed. The tests do not.

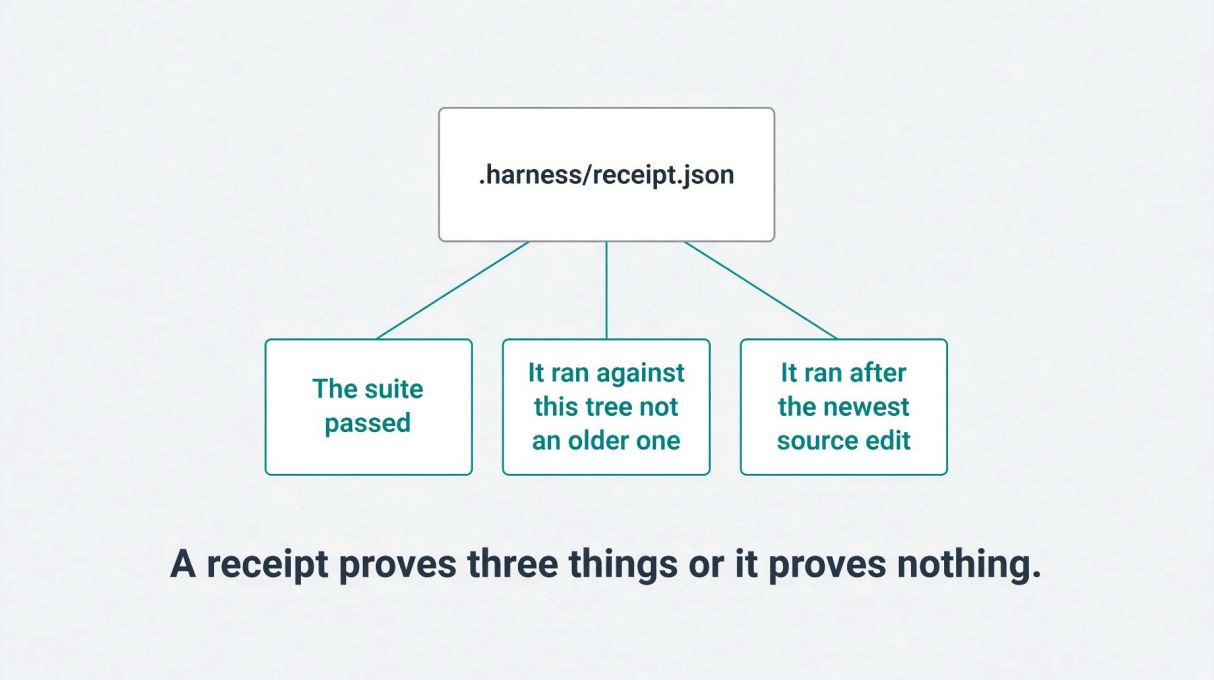
Python still owns the red gate and `gates.decide`. The model never counts its own retries.

Reading harness output

Knowing when to stop is the feature.

A receipt proves three things, or it proves nothing.

```
{"green": true,
 "tree_hash": "3da2f2dc9611...",
 "written_at": 1787720405.98,
 "report_usable": true}
```

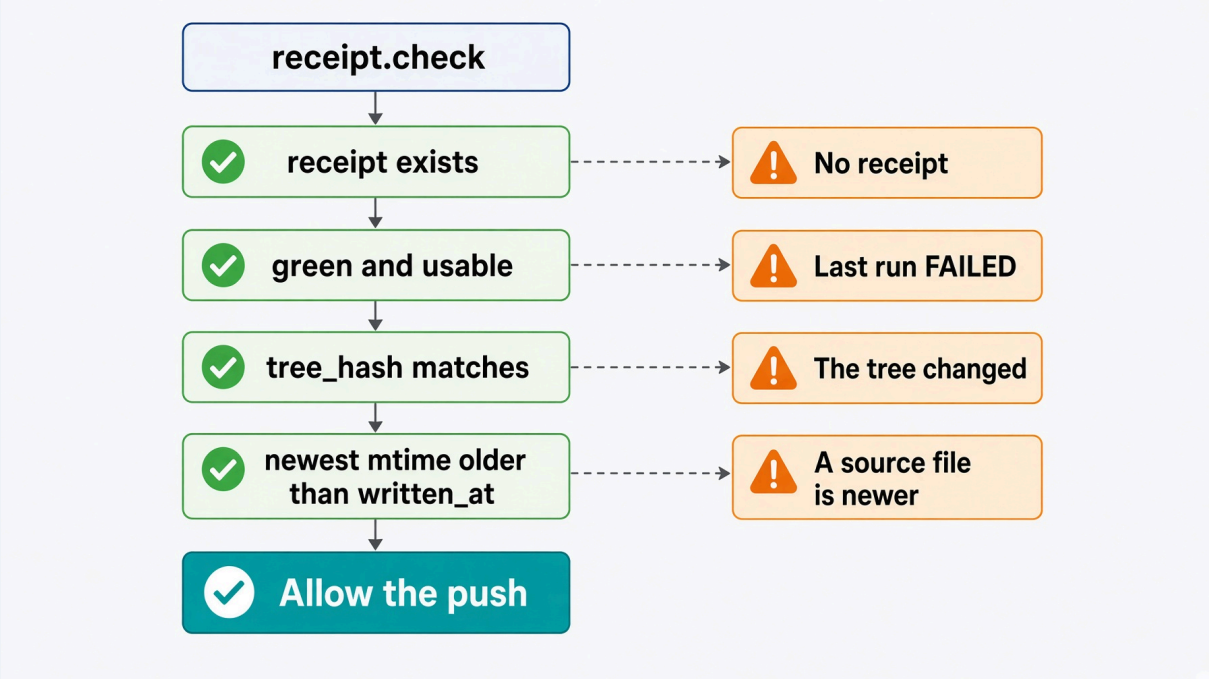


A receipt proves three things or it proves nothing.

A zero exit code with no test report is not green. It is the silent-skip bug wearing a green shirt.

`scripts/receipt.py`

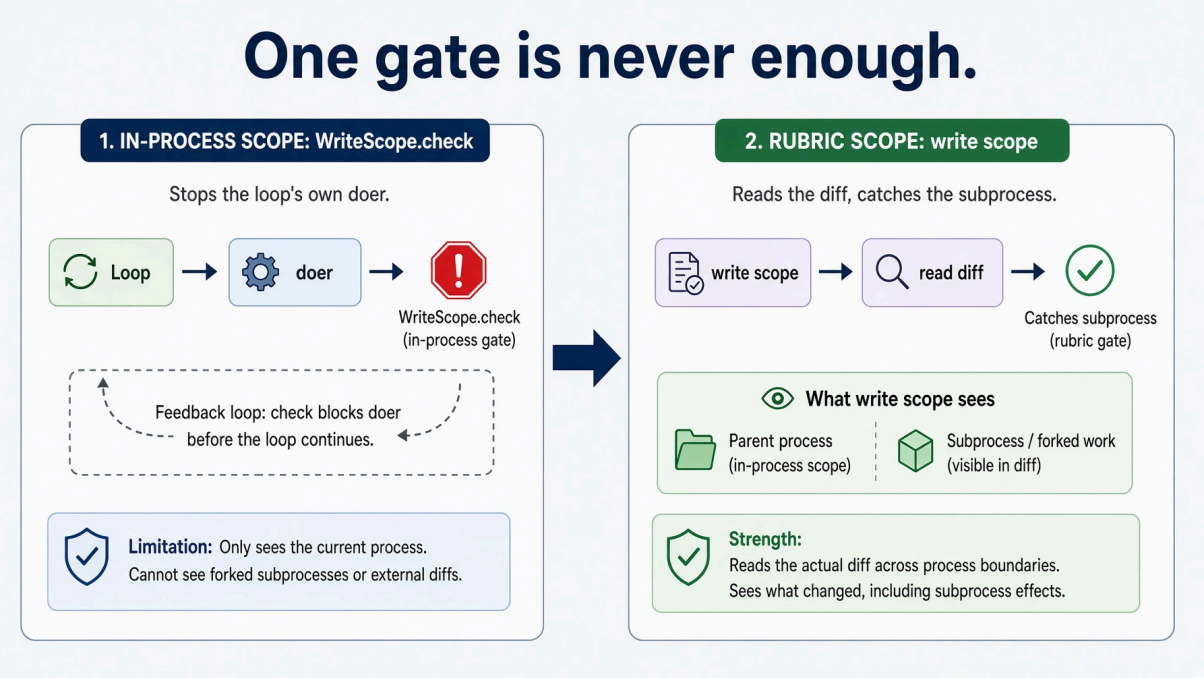
`tree_hash` and `written_at`. Stale is a refusal.



`scripts/receipt.py` · `check()`

One gate is never enough.

The write scope lives in the loop. Your agent is a subprocess.




```
in-process scope   stops the loop's own doer
rubric write_scope reads the diff, catches the subprocess
```

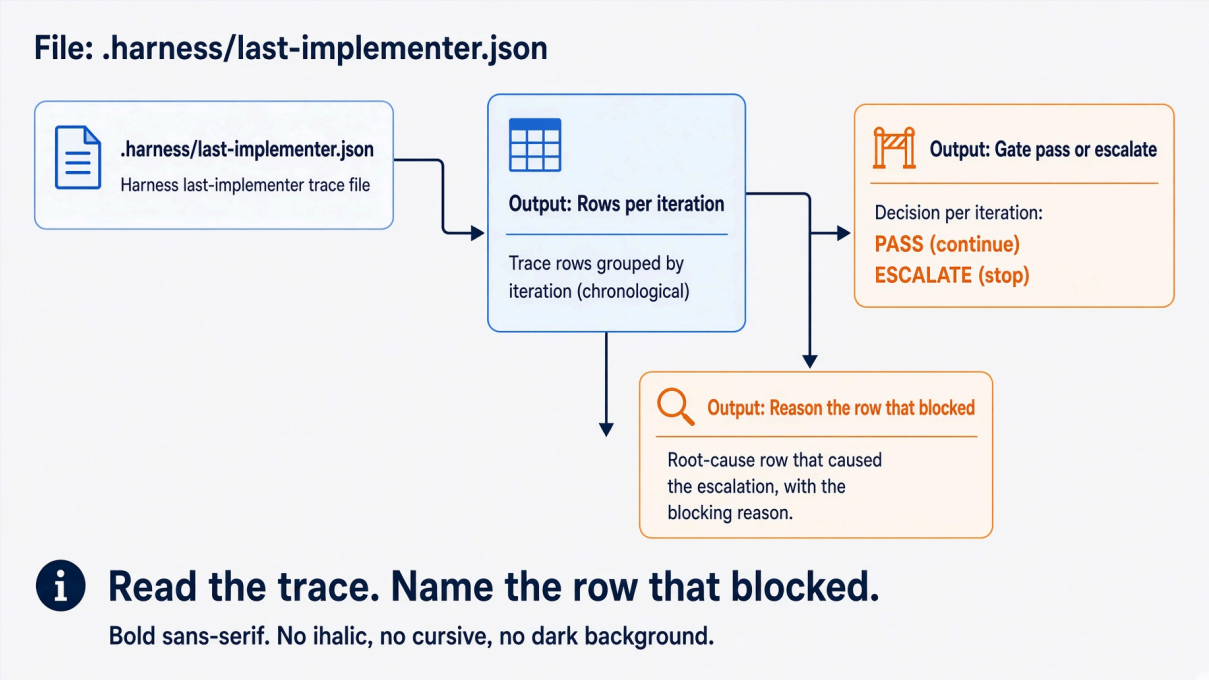
An agent that edits a test to reach green defeats the first and not the second.

Defense at one layer is a demo. Defense at two is a harness.

`labs/lab2_implementer/` · write scope in the loop and in the rubric

Read the trace. Name the row that blocked.

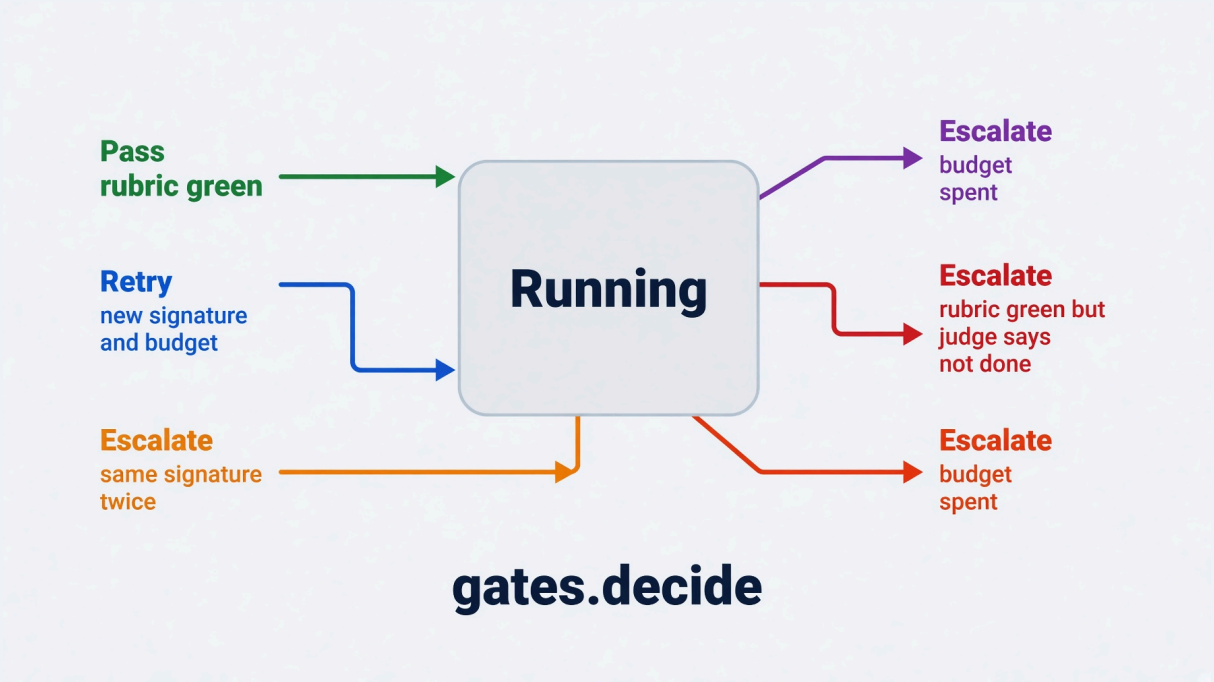
```
FAIL  coverage_floor     71.4% against a floor of 78.0%
gate: escalate
reason: the same rows failed twice: coverage_floor. Not converging.
```



- `pass` and you are done.
- `retry` and the doer gets one more scoped attempt.
- `escalate` and a human takes it.

The same rows twice means stop.

`gates.decide()`. Same signature twice is escalate.



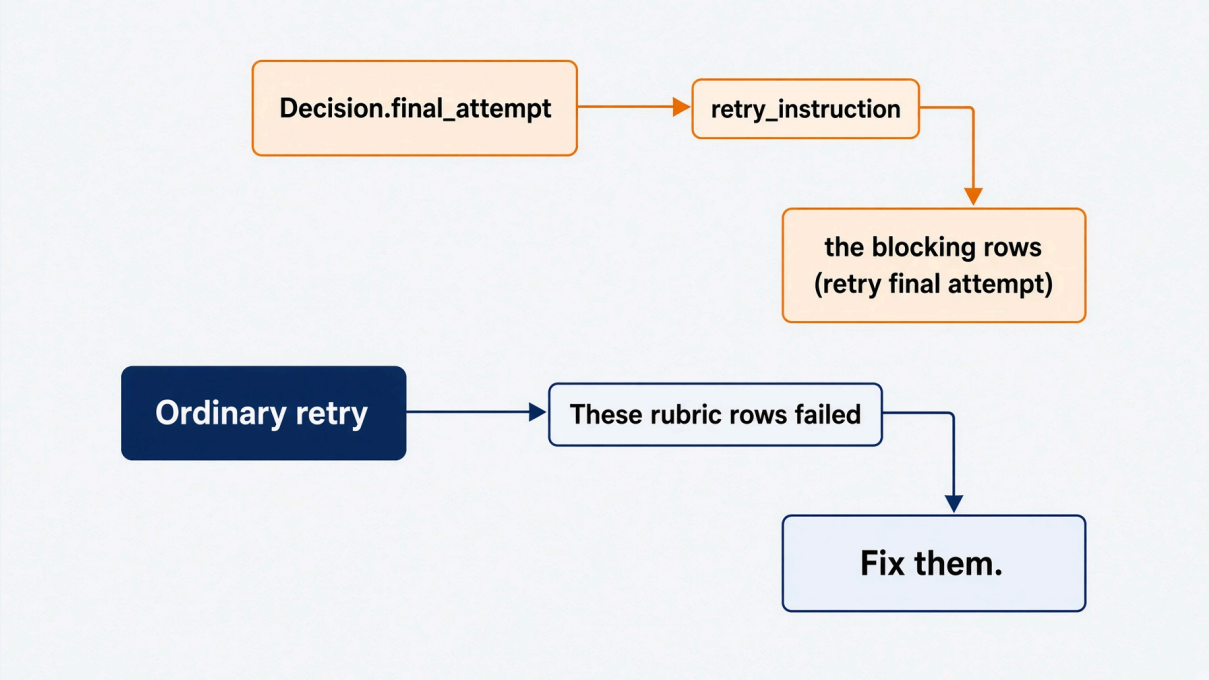
`signature` is the failed row names, not the wording.

Green rubric plus `judge_done=False` is escalate. The deterministic rows can all pass on work that misses the point.

`loops/gates.py` · `decide()`

On the final attempt, narrow the ask.

```
FINAL ATTEMPT. Fix only what blocks: tests_passed.
Do not refactor. Do not address anything else.
```



A doer that spends its last turn on a naming nit leaves the blocking row unfixed.

`loops/gates.py` · `retry_instruction()`

What you keep.

A harness that fails, iterates, and passes on its own, and refuses to ship when it should not.

The reusable evaluation harness is `harness.py` plus `loops/rubric.py`, `loops/gates.py`, and `scripts/receipt.py`.

Six lines to keep.

1. Two doers. Disjoint scope. The judge has no write method.
2. A step without a validation statement is a wish.
3. New tests must be red before code starts.
4. "The tests passed" is one row of ten.
5. Unparseable is FAIL. Same signature twice is stop.
6. A receipt proves three things, or it proves nothing.

Break. 15 minutes.

Next: the same graph, pointed at a question instead of a ticket.

Primary references for this session.

- Liu et al. Lost in the Middle. TACL 2024. arXiv:2307.03172
- Yao et al. ReAct. arXiv:2210.03629
- Ridnik, Kredo, Friedman. AlphaCodium. arXiv:2401.08500
- `loops/implementer.py` , `loops/rubric.py` , `loops/gates.py` , `loops/roles.py` , `loops/steps.py` , `loops/final_judge.py`
- `scripts/receipt.py`
- `labs/lab2_implementer/ARCHITECTURE.md`
- `solutions/sol2_implementer_deep_agents/SPEC.md`